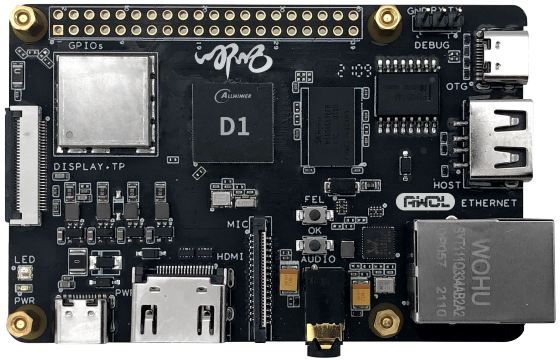
What’s the problem with D1 Linux upstream?

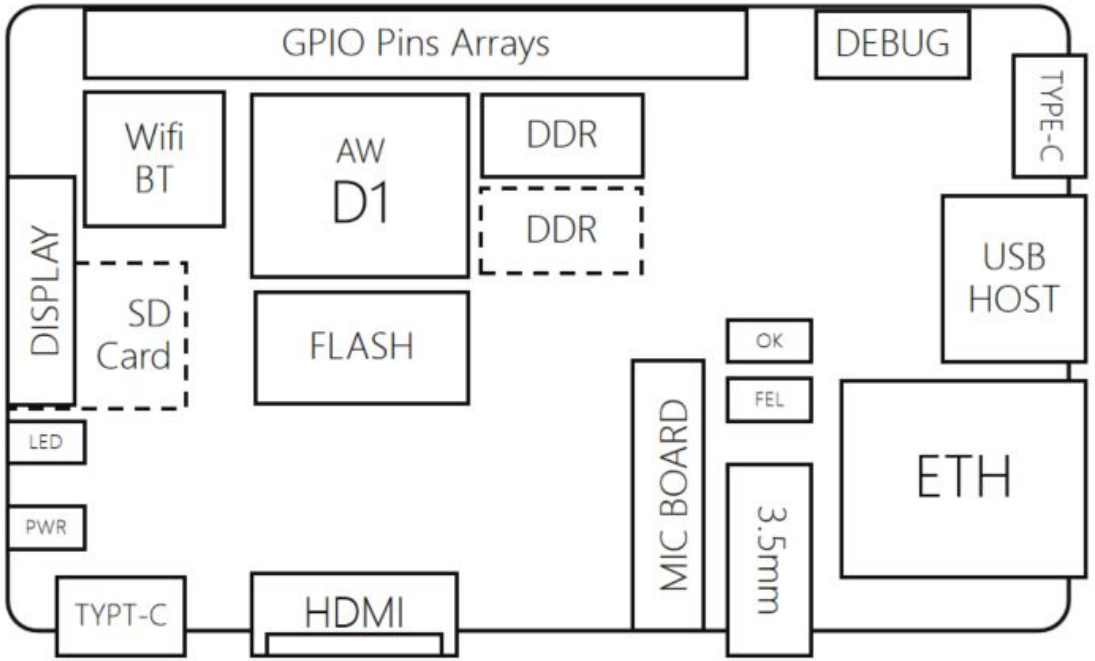
Liu Shaohua <liush@allwinnertech.com>
Fu Wei <wefu@redhat.com>
Guo Ren <guoren@kernel.org>

- BIRDVIEW OF D1
- STATUS OF D1 (linux/arch/riscv/*)
- Q & A

Allwinner D1 development board -- Nezha

Allwinner Technology





Info Source:

https://d1.docs.allwinnertech.com/

Red Hat

Allwinner D1 development board -- Nezha















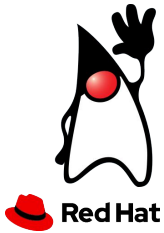










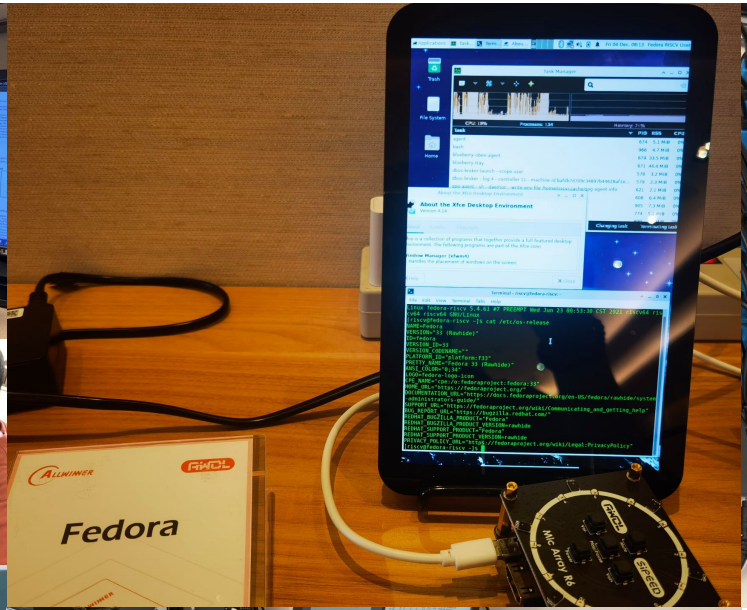


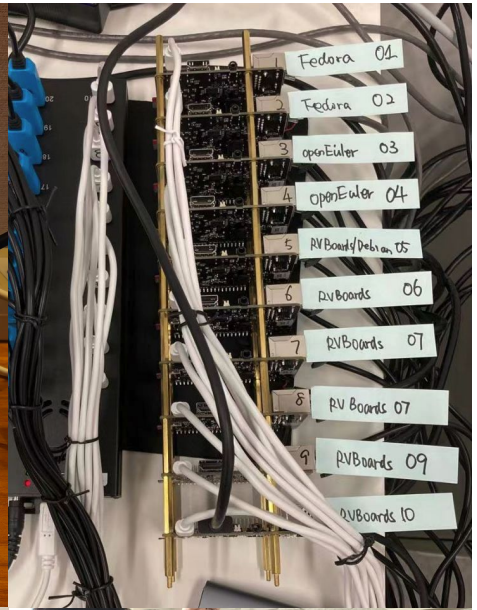
Info Source:

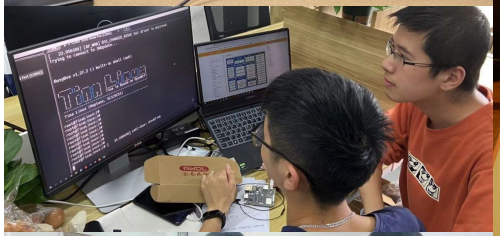
https://d1.docs.allwinnertech.com/

Allwinner D1 development board -- Nezha

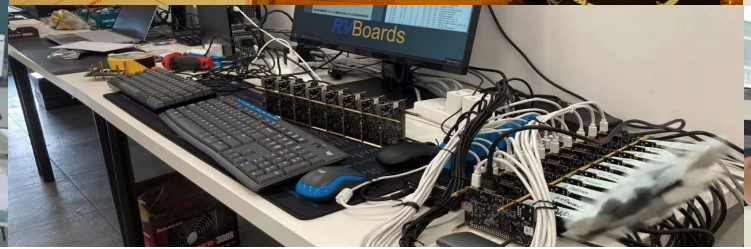














Red Hat

The Status of Nezha Firmware









Linux boot flow

boot0[1] --> OpenSBI --> U-Boot -->GRUB
-->dtb + vmlinux + initramfs --> **Linux distro**

OpenSBI(Firmware for RISC-V)

1. upstream **main** branch, **generic** platform with the nezha **dtb** file.
2. **OpenSBI 0.9 with Allwinner patches**

U-boot

The **u-boot(mainline)** with **some patches** works fine on D1 Nezha, can boot **Linux** distros.

GRUB2

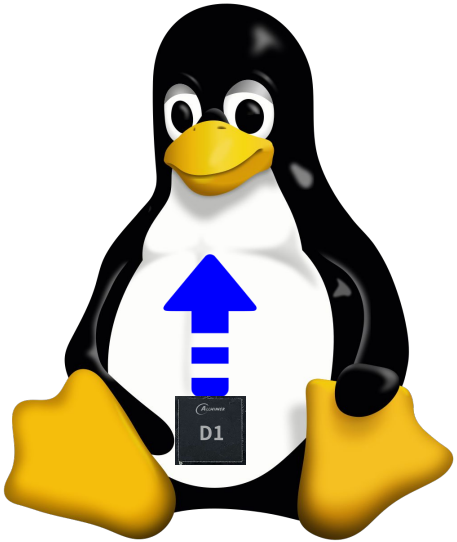
The **GRUB(mainline)** with **a few patches** works well on riscv64, can boot **Linux** distros.

FYI:

[1]Boot0(SPL) is provided by Allwinner, and it's opensource.

Red Hat

The Status of Linux kernel



Linux kernel

Linux source code from Allwinner supports all features.

RedHat's Engineer (FuWei) is using Upstream kernel with Allwinner/T-HEAD patches

We are working with T-HEAD/RedHat and opensource community on upstreaming all the patches to the **upstream kernel**.

Fedora Wiki for Allwinner Nezha D1

What is the next: UPSTREAM all features of D1



Red Hat

- BIRDVIEW OF D1
- STATUS OF D1 (linux/arch/riscv/*)
- Q & A

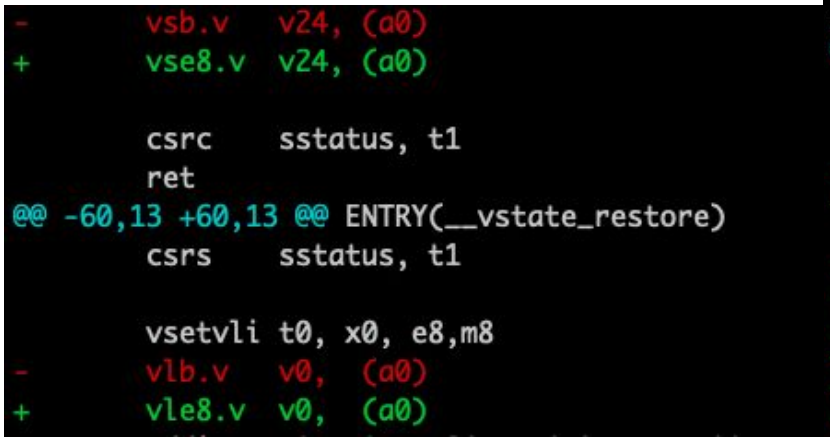
Agenda

- Vector 0.7 supported in D1
- PMU interrupt supported in D1
- I-cache synchronization acceleration in D1
- TLB hw broadcast synchronization

Fundamental patches for D1

- **custom PBMT in D1 for non-coherency SOC**
- DMA sync operations in D1

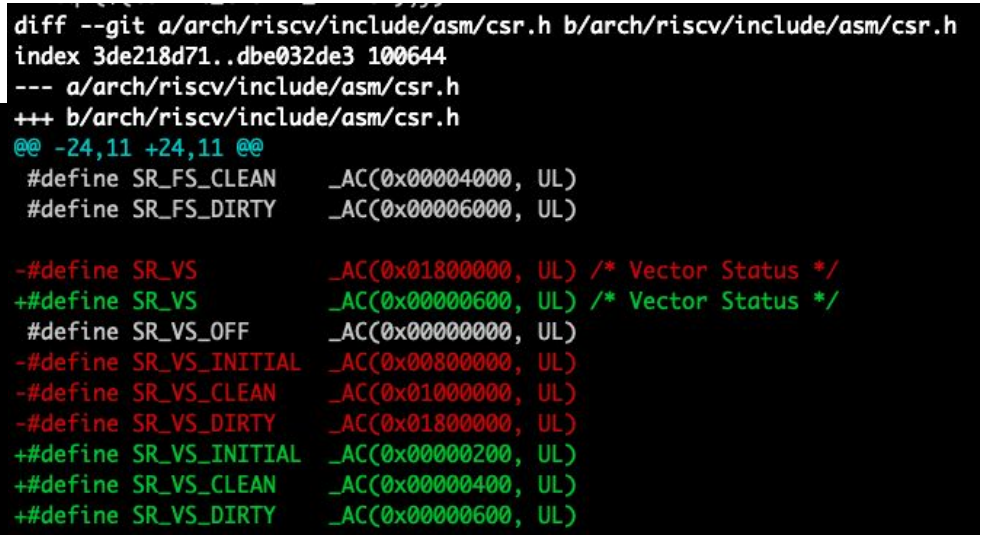
Vector 0.7.1 supported in D1



```
-       vsb.v   v24, (a0)
+       vse8.v  v24, (a0)

        csrc    sstatus, t1
        ret
@@ -60,13 +60,13 @@ ENTRY(__vstate_restore)
        csrs    sstatus, t1

        vsetvli t0, x0, e8,m8
-       vlb.v   v0,  (a0)
+       vle8.v  v0,  (a0)
```



```
diff --git a/arch/riscv/include/asm/csr.h b/arch/riscv/include/asm/csr.h
index 3de218d71..dbe032de3 100644
--- a/arch/riscv/include/asm/csr.h
+++ b/arch/riscv/include/asm/csr.h
@@ -24,11 +24,11 @@
 #define SR_FS_CLEAN     _AC(0x00004000, UL)
 #define SR_FS_DIRTY     _AC(0x00006000, UL)

-#define SR_VS           _AC(0x01800000, UL) /* Vector Status */
+#define SR_VS           _AC(0x00000600, UL) /* Vector Status */
 #define SR_VS_OFF       _AC(0x00000000, UL)
-#define SR_VS_INITIAL   _AC(0x00800000, UL)
-#define SR_VS_CLEAN     _AC(0x01000000, UL)
-#define SR_VS_DIRTY     _AC(0x01800000, UL)
+#define SR_VS_INITIAL   _AC(0x00000200, UL)
+#define SR_VS_CLEAN     _AC(0x00000400, UL)
+#define SR_VS_DIRTY     _AC(0x00000600, UL)
```

There is not so much gap between 0.7 & 1.0 for vector context switch (vlen = 128). After Greentime's patchset merged in, we would use asm(ALTERNATIVE …) of errata in vector.S to patch text. Also, the SR_VS_XXX need global variables to init.

PMU interrupt supported in D1

D1 supports “perf record” by custom CPU hw events interrupt that was implemented two years ago, you can try it with an opensbi patch + a linux pmu driver.

- Opensbi: https://github.com/c-sky/opensbi/commit/8e85bfadf5969aa916185a4bf7eef7612a8eaece
- Linux pmu driver: https://github.com/c-sky/csky-linux/blob/linux-5.10.4/drivers/perf/thead_c900_pmu.c

The coding needs to be reconstructed before upstream to follow the new infrastructure as possible as we can. Any feedbacks are welcomed.

I-cache synchronization acceleration in D1

T-HEAD C900 series provide custom icache synchronization instructions:

- icache.i(va/pa) invalidate all harts' icache line which determined by va/pa
- sync.is will flush all harts' pipeline and ensure all harts’ flush finished.

The right side vdso patch could improve the performance of OpenJDK’s JIT scenario by preventing syscall & IPI.

```
diff --git a/arch/riscv/kernel/vdso/flush_icache.S b/arch/riscv/kernel/vdso/flush_icache.S
index 82f97d67c23e..efb2d2e8a768 100644
--- a/arch/riscv/kernel/vdso/flush_icache.S
+++ b/arch/riscv/kernel/vdso/flush_icache.S
@@ -5,18 +5,39 @@

 #include <linux/linkage.h>
 #include <asm/unistd.h>
+#include <asm/cache.h>
+
+/*
+ * icache.ipa rs1
+ * | 31 - 25 | 24 - 20 | 19 - 15 | 14 - 12 | 11 - 7 | 6 - 0 |
+ *   0000001    11000      rs1       000      00000  0001011
+ *
+ * icache.iva rs1
+ * | 31 - 25 | 24 - 20 | 19 - 15 | 14 - 12 | 11 - 7 | 6 - 0 |
+ *   0000001    10000      rs1       000      00000  0001011
+ *
+ * sync.is
+ * | 31 - 25 | 24 - 20 | 19 - 15 | 14 - 12 | 11 - 7 | 6 - 0 |
+ *   0000000    11011     00000      000      00000  0001011
+ */
+#define ICACHE_IPA_X5  .long 0x0382800b
+#define ICACHE_IVA_X5  .long 0x0302800b
+#define SYNC_IS                .long 0x01b0000b

        .text
 /* int __vdso_flush_icache(void *start, void *end, unsigned long flags); */
 ENTRY(__vdso_flush_icache)
        .cfi_startproc
-#ifdef CONFIG_SMP
-       li a7, __NR_riscv_flush_icache
-       ecall
-#else
-       fence.i
+       srli    t0, a0, L1_CACHE_SHIFT
+       slli    t0, t0, L1_CACHE_SHIFT
+       addi    a1, a1, (L1_CACHE_BYTES - 1)
+       srli    a1, a1, L1_CACHE_SHIFT
+       slli    a1, a1, L1_CACHE_SHIFT
+1:     ICACHE_IVA_X5
+       addi    t0, t0, L1_CACHE_BYTES
+       bne     t0, a1, 1b
+       SYNC_IS
        li a0, 0
-#endif
        ret
        .cfi_endproc
 ENDPROC(__vdso_flush_icache)
```

TLB hw broadcast synchronization

- RISC-V Linux is now using a unified ASID mechanism between all harts, which was inspired by arm64. And this software design is very suitable for TLB hw broadcasting.

- T-HEAD c900 series provide TLB hw broadcast instructions and we could use an errata mechanism to patch text, just like SiFive's ALT_FLUSH_TLB_PAGE.

Fundamental patches for D1 bringup



Recently we rebase the D1 patchset on the latest version v5.15 and tested on qemu & hardware platform, you can try it by fedora wiki: https://fedoraproject.org/wiki/Architectures/RISC-V/Allwinner#Build_Linux_Kernel_for_D1

- The first two patches are for custom PBMT.
- The next three patches are for dma sync operations.
- The last one is just add SUNXI_SOC in Kconfig.soc

custom PBMT in D1 for non-coherency SOC

```
Svpbmt PTE format:
| 63 | 62-61 | 60-8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0
  N     MT     RSW    D   A   G   U   X   W   R   V
        ^
RISC-V
Encoding &
MemType     RISC-V Description
---------  ---------------------------------------------
00 - PMA    Normal Cacheable, No change to implied PMA memory type
01 - NC     Non-cacheable, idempotent, weakly-ordered Main Memory
10 - IO     Non-cacheable, non-idempotent, strongly-ordered I/O memory
11 - Rsvd   Reserved for future standard use
```

```
T-HEAD C9xx PTE format:
| 63 | 62 | 61 | 60 | 59-8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0
  SO   C    B    SH   RSW    D   A   G   U   X   W   R   V
  ^    ^    ^    ^
BIT(63): SO - Strong Order
BIT(62): C  - Cacheable
BIT(61): B  - Bufferable
BIT(60): SH - Shareable

MT_MASK : [63 - 59]
MT_PMA  : C + SH
MT_NC   : (none)
MT_IO   : SO
```

- Standard PBMT and D1 custom PBMT all use the highest bits to determine memory type. But the encoding is different.
- All of them would break the rule in current spec: “must be zeroed by software”
- Standard PBMT keeps the highest bits zero with normal cacheable usage, but D1 must set the highest bits with normal cacheable which needs patch linux/mm/map.c (See the first patch "pgtable: Add custom protection_map init".)
- Because of the normal cacheable type’s highest bits non-zero, D1 also needs a very early errata_init before setup_vm.

riscv: pgtable: Add custom protection_map init (Abandoned)

D1 needn’t this patch now, we could init protection_map[] directly.

```
diff --git a/arch/riscv/mm/init.c b/arch/riscv/mm/init.c
index c0cddf0fc22d..f74c09a82870 100644
--- a/arch/riscv/mm/init.c
+++ b/arch/riscv/mm/init.c
@@ -589,10 +589,32 @@ static void __init create_fdt_early_page_table(pgd_t *pg
        dtb_early_pa = dtb_pa;
 }

+static void __init setup_protection_map(void)
+{
+       protection_map[0]  = __P000;
+       protection_map[1]  = __P001;
+       protection_map[2]  = __P010;
+       protection_map[3]  = __P011;
+       protection_map[4]  = __P100;
+       protection_map[5]  = __P101;
+       protection_map[6]  = __P110;
+       protection_map[7]  = __P111;
+       protection_map[8]  = __S000;
+       protection_map[9]  = __S001;
+       protection_map[10] = __S010;
+       protection_map[11] = __S011;
+       protection_map[12] = __S100;
+       protection_map[13] = __S101;
+       protection_map[14] = __S110;
+       protection_map[15] = __S111;
+}
+
 asmlinkage void __init setup_vm(uintptr_t dtb_pa)
 {
        pmd_t __maybe_unused fix_bmap_spmd, fix_bmap_epmd;

+       setup_protection_map();
+
        kernel_map.virt_addr = KERNEL_LINK_ADDR;

 #ifdef CONFIG_XIP_KERNEL
diff --git a/mm/mmap.c b/mm/mmap.c
index 88dcc5c25225..55b9da1f8b56 100644
--- a/mm/mmap.c
+++ b/mm/mmap.c
@@ -100,10 +100,14 @@ static void unmap_region(struct mm_struct *mm,
  *                                                             w: (no) no
  *                                                             x: (yes) yes
  */
+#ifdef CONFIG_ARCH_HAS_PROTECTION_MAP_INIT
+pgprot_t protection_map[16] __ro_after_init;
+#else
 pgprot_t protection_map[16] __ro_after_init = {
        __P000, __P001, __P010, __P011, __P100, __P101, __P110, __P111,
        __S000, __S001, __S010, __S011, __S100, __S101, __S110, __S111
 };
+#endif
```

riscv: errata: pgtable: Add custom Svpbmt ...

- Add "__riscv_pbmt" into all _PAGE_XXX's definitions in pgtable.h.
- Add apply_errata_setup_vm

Here are issues need discussion:

1. How to determine standard/custom/none PBMT?
2. How to compatible with global dma pool style?

```
+/*
+ * This is called very early form setup_vm in the boot process.
+ */
+void __init apply_errata_setup_vm(void)
+{
+       riscv_fill_cpu_mfr_info();
+
+       switch (cpu_mfr_info.vendor_id) {
+#ifdef CONFIG_ERRATA_THEAD
+       case THEAD_VENDOR_ID:
+               thead_errata_setup_vm(cpu_mfr_info.arch_id,
+                                     cpu_mfr_info.imp_id);
+               break;
+#endif
+       default:
+               break;
+       }
+}
diff --git a/arch/riscv/include/asm/vendorid_list.h b/arch/riscv/includ
index 9d934215b3c8..bdfce1a4b42b 100644
--- a/arch/riscv/include/asm/vendorid_list.h
+++ b/arch/riscv/include/asm/vendorid_list.h
@@ -6,5 +6,6 @@
 #define ASM_VENDOR_LIST_H

 #define SIFIVE_VENDOR_ID       0x489
+#define  THEAD_VENDOR_ID       0x5b7

 #endif
diff --git a/arch/riscv/mm/init.c b/arch/riscv/mm/init.c
index f74c09a82870..48cdc8ddbdba 100644
--- a/arch/riscv/mm/init.c
+++ b/arch/riscv/mm/init.c
@@ -613,6 +613,8 @@ asmlinkage void __init setup_vm(uintptr_t dtb_pa)
 {
        pmd_t __maybe_unused fix_bmap_spmd, fix_bmap_epmd;

+       apply_errata_setup_vm();
+
        setup_protection_map();

        kernel_map.virt_addr = KERNEL_LINK_ADDR;
@@ -877,3 +879,8 @@ int __meminit vmemmap_populate(unsigned long start,
        return vmemmap_populate_basepages(start, end, node, NULL);
 }
 #endif
+
+#ifdef CONFIG_64BIT
+struct __riscv_pbmt_struct __riscv_pbmt __ro_after_init;
+EXPORT_SYMBOL(__riscv_pbmt);
+#endif
```

DMA sync operations in D1

D1 has custom dcache clean/invalidate operation instructions for DMA synchronization

- Cache line based
- Could be used in supervisor mode

(If future CMO instruction also bases on the cache line, we could use “asm(ALT...)” to patch text on it.)

```
+#ifdef CONFIG_RISCV_DMA_NONCOHERENT
+/*
+ * dcache.ipa rs1 (invalidate)
+ * | 31 - 25 | 24 - 20 | 19 - 15 | 14 - 12 |
+ *   0000001    01010      rs1       000
+ *
+ * dcache.cpa rs1 (clean)
+ * | 31 - 25 | 24 - 20 | 19 - 15 | 14 - 12 |
+ *   0000001    01001      rs1       000
+ *
+ * dcache.cipa rs1 (clean then invalidate)
+ * | 31 - 25 | 24 - 20 | 19 - 15 | 14 - 12 |
+ *   0000001    01011      rs1       000
+ *
+ * sync.s
+ * | 31 - 25 | 24 - 20 | 19 - 15 | 14 - 12 |
+ *   0000000    11001     00000      000
+ */
+#define DCACHE_IPA_A0  ".long 0x02a5000b"
+#define DCACHE_CPA_A0  ".long 0x0295000b"
+#define DCACHE_CIPA_A0 ".long 0x02b5000b"
+
+#define SYNC_S         ".long 0x0190000b"
```

At the End

The most I worried about in D1 is the non-zero-highest bits for Normal Cacheable Memory Type of custom PBMT.

- A small modification to protection_map init in mm/mmap.c
- Introduce a new apply_errata_setup_vm() for errata in arch/riscv

Q & A
Thank you